

Memory Safety on a Microcontroller

What Embedded C++ Teams Can Actually Do Today

DARIJO TOPIC

Memory Safety on a Microcontroller

What Embedded C++ Teams Can Actually Do Today

Darijo Topić · CppCon 2026 · Embedded Track · Friday 18 September

Cortex-M33 · no MMU

Zephyr 4.4

GCC 14.3

C++23 today · C++26 preview

every number: Compiler Explorer

Embedded C++ is not just C++

The job is silicon, a toolchain and a hardware team — the language is one component of it

The language

C++17 / 20 / 23 — the part CppCon talks about. One component of the system, not the system.

The silicon

Datasheet, errata, registers, clocks, DMA, the memory map. You read them like a spec.

The toolchain

The compiler is a component with a datasheet — [intro.compliance]. Linker script, RTOS, on-chip debug.

The people

HW / EE team, schematics, board bring-up, the certification body. Firmware is a collaboration.

We came to C++ from the hardware side: we read datasheets for a living and treat the compiler as a component that has none. It does — and half this talk is reading it like an errata sheet.

You are not on a PC — or even a Linux SOM

The constraints that make every later slide necessary

	PC / server	Linux SOM	Microcontroller
RAM	16–128 GB	256 MB – several GB	tens to hundreds of KB
Memory model	virtual, per process	virtual, MMU	one flat physical space; address 0 is readable
Supervisor	OS restarts a crash	OS restarts a crash	you are the OS — nothing restarts you
Updates	apt, instantly	OTA images	maybe never; a brick is a truck roll
Timing	best-effort	soft real-time	hard deadlines in microseconds
On failure	crash = mitigation	crash = mitigation	crash = physical harm
Debugging	gdb, ASan, core dumps	mostly the same	a fault handler and an oscilloscope

Every mitigation in the memory-safety literature assumes column one or two. This talk lives in column three.

What are we trying to keep safe?

A microcontroller has more failure surfaces than the C++ abstract machine names.

SPATIAL

Outside the intended region

Out-of-bounds access, wrong length, buffer overflow.

TEMPORAL

Outside the intended lifetime

Use-after-scope, stale ownership, DMA outliving its buffer.

STATE / TYPE

Bad state or type assumptions

Uninitialized data or invalid object assumptions. Arithmetic can manufacture a bad bound.

EMBEDDED

CPU is not the only writer.

ISR, DMA, peripherals and the memory system participate.

THE GOAL

Make unintended memory operations impossible, detectable, or observable before they become uncontrolled device behavior.

That is the lens for every technique that follows.

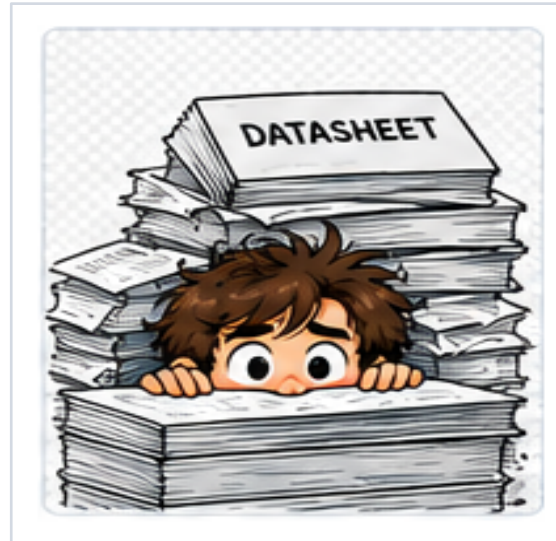
Once upon a firmware...

Four panels from one early career — where the confessions in this talk come from



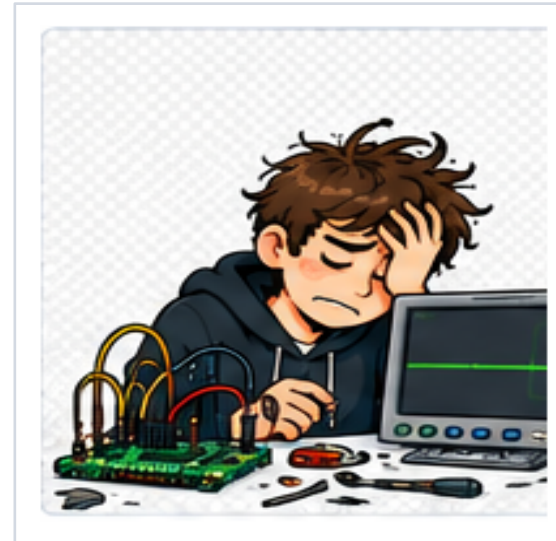
Day one

A shiny new board. Zero doubts.



Week one

The datasheet avalanche.



Day nine

It won't boot. The scope shows nothing.



Ever since

The advice that doesn't fit 64 KB.

Watch the corner through the rest of the talk — the confessions are this character's beats.

Most memory-safety advice assumes a computer you don't have

The hosted mitigations, and what a Cortex-M actually gives you

Mitigation	Assumes	You have
Isolation, guard pages, ASLR	MMU, virtual memory	No Cortex-M has an MMU. An MPU: 8 regions, no translation
W⊕X on every page	Page permissions	SRAM is executable by default; XN is an MPU bit somebody sets
A process boundary	An OS handing out address spaces	One flat address space; a thread is a stack, not a boundary
AddressSanitizer	Shadow memory 1:8, redzones, quarantine	Tens of KB of SRAM on most parts — ASan is a host tool here
Memory tagging (MTE)	64-bit pointers with a spare byte	32-bit pointers. Structurally never coming to M-profile
"Just rewrite it"	A team, a budget, no certification	200k lines, field-proven, possibly certified

A hosted memory bug lands inside a process, behind ASLR, on a W⊕X page. Yours lands in a flat address space with none of that.

The number that should bother you

1,797 real Cortex-M firmware images, statically analysed — Tan et al., USENIX WOOT 2024

0

of 1,797 set the MSPLIM/PSPLIM stack-limit registers Armv8-M gives them for free

1

image with stack canaries

5

use the Arm MPU — 0.28 %

"Without privilege separation, any bug can be critical and compromise the entire system."

State the sampling first: 81 % Nordic silicon, consumer-BLE-heavy, automotive and medical essentially absent. In that corpus the finding survives intact.

The problem is not only that our tools are weaker. It is that we do not switch on the protection we already have.

Find out what you actually have

The MPU is optional, its region count is a vendor choice — and MSPLIM depends on your half of Armv8-M

```
// target_traits.hpp – the only file allowed to name __ARM_*
namespace target {
inline constexpr int arch = __ARM_ARCH; // 6, 7, 8
inline constexpr bool mainline = (__ARM_ARCH_ISA_THUMB == 2);
inline constexpr bool pacbti = has(__ARM_FEATURE_PAUTH);
}
// a value with a type and a name – no #ifdef soup downstream
```

```
// 1. a wrong target is a build error with a sentence
static_assert(target::arch >= 8 && target::mainline,
    "this driver reads MSPLIM; Armv8-M Baseline has none");
// 2. per-target code, both branches type-checked
if constexpr (target::mainline) enable_stack_limit();
else configure_mpu_guard();
```

Find out what you actually have

The MPU is optional, its region count is a vendor choice — and MSPLIM depends on your half of Armv8-M

static_assert answers for the architecture you asked for.
Only a register answers for the part you were given.

```
static_assert(target::mainline);           // the ISA — settled at build time
__ASSERT(mpu_regions() >= 2, "no MPU: the stack guard is not there");
                                           // MPU_TYPE.DREGION — at boot
```

A -mcpu=cortex-m33 line is a request, not a safety argument.

There is no compile-time way to know you have an MPU — ACLE has no macro for it.

Five bug classes, and the one the C++ memory model has no word for

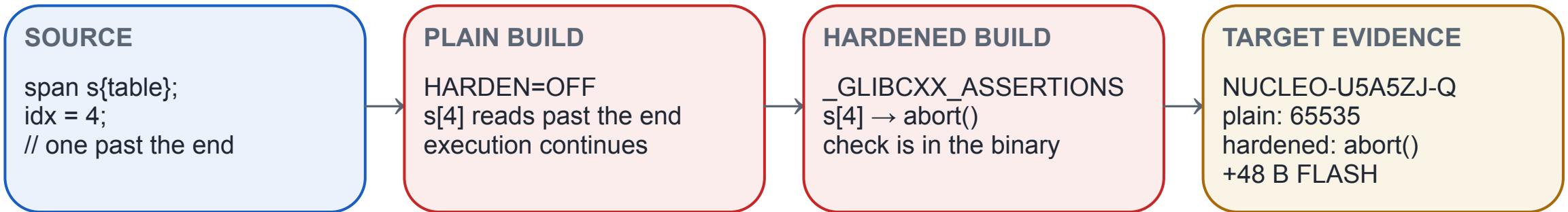
Class	What it is	How it shows up in firmware
Spatial	Access outside an object's bounds	Length field from the radio used as an index
Temporal	Access outside an object's lifetime	A DMA engine writing into a popped stack frame
Initialization	Reading a value never written	Uninitialized struct member used as a length
Type	Reinterpreting memory as the wrong type	reinterpret_cast over a byte buffer; union punning
Arithmetic (precursor)	count × size wraps → undersized allocation	BadAlloc: 25+ CVEs, one idiom, ~two dozen RTOSes

And the one that is ours: an interrupt.

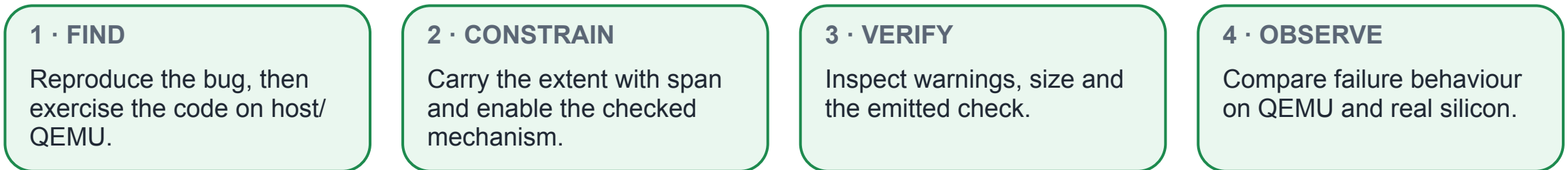
Neither a thread nor a signal handler, so neither set of guarantees applies. 23,000 lines of Core Guidelines never contain the word "interrupt"; CP.201 is an empty stub.

One real bug, end to end

examples/02_span_hardenend — same source, two builds, two outcomes.



Now carry this example through the talk



Same source-level bug. Different mechanisms. This is the thread — not a fifth technical section.

Nobody with authority said "rewrite it in Rust"

What the regulators actually asked for — and the asymmetry they did not think about

The mandate

- CISA/FBI, Oct 2024: publish a memory-safety roadmap by 1 Jan 2026. A document, not a rewrite. The date has passed.
- CISA/NSA, June 2025: "completely rewriting existing codebases is often impractical" — then lists bounds checking, compiler options, sanitizers.
- EU CRA Art. 14: vulnerability reporting on a 24 h / 72 h / 14 d clock — live since 11 September 2026.
- No binding instrument anywhere names a language.

The asymmetry

On a hosted system, crashing is a mitigation. Detect, abort, let the supervisor restart: the attack is defeated and the cost is a blip.

On a device that is heating, moving or dosing something, crashing is the harm. Every hardening mechanism in this talk terminates. What the device does in the 200 ms afterwards belongs to your safety case.

With great power comes great responsibility: Week 0 of the checklist is a design decision, not a flag.

Memory safety $\neq$ security $\neq$ functional safety

The same memory bug can matter for different reasons — and the response is not identical.

MEMORY SAFETY

Prevent or detect unintended memory operations.

Question: can this access happen?

SECURITY

Assume an adversary may choose inputs or timing.

Question: can this become an exploit?

FUNCTIONAL SAFETY

Control the consequences of failure in the physical system.

Question: what does the device do next?

One failure can sit in all three boxes.

A bounds check may prevent corruption. A fault handler may contain the consequence. A safety case decides whether the remaining behavior is acceptable.

How every technique in this talk gets judged

Three axes — the same three at the end of each layer, and on the checklist you take home

1 · Code churn

How many lines change, and whose.
Zero, new code only, or a refactor.

2 · Cost on target

Flash bytes and cycles — measured,
on this compiler, at your shipping -O
level.

3 · Bug class prevented

Which failure from the table two slides
ago. Not "safety".

A technique that cannot answer all three does not go on a roadmap.

Every cost figure in this deck was compiled on Compiler Explorer with bare-metal ARM GCC 14.3.0 — the Zephyr SDK version. Cycle figures need the board and are not on any slide. Godbolt = what the compiler emits; the Nucleo = what the silicon does; when they disagree, the silicon wins.

Two habits that do the heavy lifting

Before the flags and the techniques — the two that change the most

Test where the sanitizers live

- The bugs are in the logic — the parser, the state machine, the arithmetic — not in the register pokes.
- Split that logic from a thin, audited hardware layer. Compile it for the host and run ASan / MSan / UBSan / a fuzzer there.
- Your strongest memory-safety tool is a build that never touches the board.

Make failure observable

- There is no debugger attached in the field. A fault must say where it happened.
- A handler that dumps the stacked PC/LR and registers; DWT; RTT / semihosting on the bench; stack high-water in telemetry.
- A silent reset teaches you nothing — and hides the bug you shipped.

Neither costs a library. Together they decide whether everything after this slide is measurable.

Layer 1

Free wins: change nothing, enable everything

The language has no safety features to switch on. Your implementation already knows things about your program it is under no obligation to tell you. These are the flags that make it talk.

≈ 11 min

The warning you think you have

-Wconversion does not imply -Wsign-conversion in C++. It does in C.

```
void put(std::uint8_t);  
void f(int n, std::int16_t s) {  
    put(n); // -Wconversion: warns  
    std::uint32_t u = s; // -Wconversion: silent  
    put(static_cast<std::uint8_t>(u));  
}
```

GCC manual, Warning Options

"...sign-conversion warnings are disabled by default in C++ unless -Wsign-conversion is explicitly enabled."

- Firmware is uint8_t, int16_t and register arithmetic — exactly where the bugs live. Zephyr never passes it.
- Monday: -Wsign-conversion by name, -Werror, baseline the existing warnings, enforce zero new. Warnings you do not treat as errors are warnings you do not read.

godbolt.org/z/fobnz9K6M — ARM GCC 14.3.0, Cortex-M33

Your debug build warns less than your release build

The checks you never saw, because you review warnings at -O0

```
int f(int c) {  
    int x;  
    if (c > 3) x = h();  
    if (c > 1) return x;  
    // -O0: silent · -Og/-Os: warning  
    return 0;  
}
```

```
std::array<int,4> a{}; return a[5]; // -O0 silent · -Os warns  
std::span<int,4> s{a}; return s[5]; // -O0 silent · -Os warns  
std::span<int> d{a}; return d[5]; // silent: backing size opaque  
std::memcpy(buf16.data(), src, 32); // warns even at -O0
```

godbolt.org/z/M1qEovvjrr (-Wmaybe-uninitialized)

godbolt.org/z/xa9cq1h3T (-Warray-bounds)

Your debug build warns less than your release build

The checks you never saw, because you review warnings at -O0

Access (ARM GCC 14.3.0, -Wall -Wextra)	-O0	-O1	-Os
<code>int x; ...return x;</code>	silent	warns	warns
<code>int a[4]; a[5] · std::array<int,4>[5] · span<int,4>[5]</code>	silent	silent	-Warray-bounds
<code>std::span<int> (dynamic extent) [5]</code>	silent	silent	silent

The warnings were in -Wall all along; the optimizer's dataflow produces them.

Fixed extent: caught statically. Dynamic extent: silent when the backing size is opaque — exactly the case the hardened library covers.

Review your warnings at the level you ship, not the level you debug at

godbolt.org/z/M1qEovvjrr (-Wmaybe-uninitialized)

godbolt.org/z/xa9cq1h3T (-Warray-bounds)

-fhardened does not work on your target

and it withholds the one piece of itself that would have

```
$ arm-none-eabi-g++ -fhardened -Os x.cpp
cclplus: warning: '-fhardened' not supported
        for this target

// gcc/opts.cc: flag_hardened = 0; → the whole
// umbrella is off, not the unsupported parts
```

What the umbrella contains (GCC 14 manual)

```
-D_FORTIFY_SOURCE=3
-D_GLIBCXX_ASSERTIONS
-ftrivial-auto-var-init=zero
-fstack-protector-strong
-fstack-clash-protection
-fcf-protection=full
-fPIE -pie -Wl,-z,relro,-z,now
```

- `-D_GLIBCXX_ASSERTIONS` is a macro read by header code. No target dependency whatsoever — withheld with the rest.
- `-ftrivial-auto-var-init=zero` is the wrong value for a device: a zero pointer reads the vector table instead of faulting. Prefer `=pattern (0xFE)`: +10 bytes per function with locals.
- Monday: ask for the portable pieces by name.

godbolt.org/z/dhv5rsEEEx

Your sanitizer is not looking at your C++

UBSan runs on a Cortex-M. On Zephyr, it is applied to C only.

```
# zephyr v4.4.0, root CMakeLists.txt
if(CONFIG_UBSAN)
  zephyr_compile_options(
    $$<COMPILE_LANGUAGE:C>:...sanitizer_undefined>
    zephyr_link_libraries(...sanitizer_undefined)    # unguarded
    ...
# there is no CXX arm. Warnings, -Werror, canaries,
# -fno-strict-aliasing: all have both. UBSan has one.
```

What you get

CONFIG_UBSAN=y is accepted. The runtime is linked. The image grows. menuconfig says "enabled". The kernel and HAL are instrumented. Your C++: not checked.

Your sanitizer is not looking at your C++

UBSan runs on a Cortex-M. On Zephyr, it is applied to C only.

```
# your app's CMakeLists.txt — the one-line fix
target_compile_options(app PRIVATE
  ${$<COMPILE_LANGUAGE:CXX>:
    -fsanitize=undefined
    -fsanitize-trap=undefined>)
```

- Second hole: `-fsanitize=vprr` — the only check about C++ object identity — needs RTTI. `-fno-rtti` drops it silently, and `-fno-rtti` is Zephyr's default.
- Two builds: `UBSAN_LIBRARY` on `picolibc` for the bench, `UBSAN_TRAP` to ship.

The absence of a check produces no error, no warning and no line in the build log.

Probe: `ninja -t commands | grep fsanitize`

godbolt.org/z/9GdrTKadn — `vprr: 1 handler with RTTI, 0 with -fno-rtti`

std::byte silently loses your stack canary

The only place in this talk where modern-C++ advice removes a mitigation you already had

```
// gcc/cfgexpand.cc
// stack_protect_classify_type
if (t == char_type_node
    || t == signed_char_type_node
    || t == unsigned_char_type_node)
/* compare length with
   ssp-buffer-size */
```

std::byte is enum class byte : unsigned char — an enumeration, not a character type. Change uint8_t frame[64] to std::byte frame[64], as the Core Guidelines ask — no behavioural difference, no diagnostic — and the canary on that function is gone.

Local (ARM GCC 14.3.0, -O1)	-fstack-protector	-strong
char buf[64]	✓	✓
std::uint8_t buf[64]	✓	✓
std::array<std::uint8_t, 64>	✓	✓
std::byte buf[64]	✗	✓
std::array<std::byte, 64>	✗	✓
std::uint32_t buf[16]	✗	✓

A stack canary is a tripwire planted between your buffer and the return address, checked before every return — it turns a stack smash into a controlled panic instead of a hijacked return. std::byte silently removes it (it's an enum, not a char), so use -fstack-protector-strong, which guards every local array regardless of type.

godbolt.org/z/33Mz7a4xK

One line bounds-checks every container in Layer 2

-D_GLIBCXX_ASSERTIONS — zero source churn, works under -fno-exceptions, ships this week

```
// span<int,4>::operator[], -Os
// plain:
ldr  r0, [r0, r1, lsl #2]
bx   lr
// -D_GLIBCXX_ASSERTIONS:
    cmp    r1, #3
    push   {r3, lr}
    bls    .L2
    bl     abort ← no strings
.L2: ldr    r0, [r0, r1, lsl #2]
    pop    {r3, pc}
```

Accessor	plain	hardened
span<int,4>[i]	6 B	16 B (+10)
span<int>[i]	14 B	24 B (+10)
array<int,4>[i]	6 B	16 B (+10)
string_view[i]	12 B	22 B (+10)
optional<int>::operator*	4 B	14 B (+10)
raw int* [i] (control)	6 B	6 B

- +10 bytes per checked site. Bare-metal libstdc++ is built non-verbose: cmp, branch, bl abort — no file, no line, no .rodata.
- On Zephyr, abort() is printk + k_panic(): a logged panic into the weak k_sys_fatal_error_handler() you own.
- libc++: -D_LIBCPP_HARDENING_MODE=_LIBCPP_HARDENING_MODE_FAST — one 16-bit udf per site.

godbolt.org/z/MraE5rac6 (hardened) godbolt.org/z/Ed54ve9PW (plain)

Layer 1 scorecard

Change nothing in your source. Read what you already pass. Then add these.

Technique	Code churn	Cost on target	Bug class
<code>-Wsign-conversion -Wextra -Werror</code> , baseline	zero	0 B	conversions, dangling refs
Review warnings at your shipping <code>-O</code> level	zero	0 B	uninit reads, constant-index overruns
<code>_GLIBCXX_ASSERTIONS</code> / <code>_LIBCPP_HARDENING_MODE</code>	zero (a flag)	+10 B per checked site	spatial — every §5 container
UBSan on target + the CXX line on Zephyr	one build-file line	checks only (trap mode)	signed overflow, null, misalignment
<code>-fstack-protector-strong</code> (CANARIES_STRONG)	zero	a canary in more functions	frame-local overrun
<code>-ftrivial-auto-var-init=pattern</code>	zero	+10 B per function with locals	uninitialized reads
<code>HW_STACK_PROTECTION</code> (MSPLIM/MPU guard)	two config lines	0 B / one MPU region	stack overflow
<code>clang-tidy</code> , <code>-fanalyzer</code> (C only), host ASan	CI work	0 B on target	OOB, UAF, leaks

Byte figures: Compiler Explorer, ARM GCC 14.3.0, Cortex-M33, -Os, per function.

The compiler can help.

But the binary is the evidence.

WARN → BUILD → INSPECT → VERIFY

Layer 2

Code-level modern C++ — where it earns its keep

New code only: you do not have to touch the legacy for the curve to bend. But the technique is not the mechanism — every one of these has a second half everybody forgets.

≈ 19 min

= delete operator new does not work

Here is what everyone writes, here is the paragraph that says it cannot, here is what does

```
void* operator new(std::size_t) = delete; // don't  
int* f() { return new int(1); } // another TU
```

- Ill-formed. [dcl.fct.def.delete]/4: a deleted definition shall be the first declaration. [basic.stc.dynamic]/2: the global allocation functions are implicitly declared in every translation unit. Yours is never first.
- Clang rejects it: error: deleted definition must be first declaration — off tiarmclang, ATfE, STARM-Clang, Silabs LLVM.
- GCC warns, and links clean. A deleted definition emits no symbol; the allocating TU never saw it; the linker resolves `_Znwj` from `libstdc++`. `std::vector + new int` linked, static and dynamic.

godbolt.org/z/v5Ws66z3M (clang rejects it)

Define all twenty — against a symbol nobody defines

One translation unit, project-wide effect, enforced at link time

```
// no_heap.cpp
extern "C" [[noreturn]] void no_heap();    // declared, never defined
void* operator new (std::size_t)          { no_heap(); }
void* operator new[] (std::size_t)        { no_heap(); }
void* operator new (std::size_t, std::align_val_t) { no_heap(); }
void* operator new (std::size_t, const std::nothrow_t&) noexcept {no_heap();}
void operator delete(void*) noexcept      { no_heap(); }
// ... 15 more: sized, aligned, nothrow, [] forms.
// Never the four placement forms: not replaceable.
// Any std::vector, anywhere → undefined reference to 'no_heap'
```

Twenty, not eight

The over-aligned forms are the ones an alignas(32) DMA buffer takes — the eight-function version waved exactly that through.

A virtual destructor references operator delete from its vtable even if nothing is ever allocated — the second reason to define rather than delete.

Now a `std::vector` in a header, a `std::string` added in someone's second week, an algorithm that allocates internally: a build failure that names itself, on every build, forever.

godbolt.org/z/48E31fP69 (the twenty, GCC 14.3)

On Zephyr this is a correctness fix, not a discipline one

The default operator new returns null — and the standard says it may not

```
// zephyr v4.4.0 lib/cpp/minimal/cpp_new.cpp
// (the DEFAULT library)
NODISCARD void* operator new(size_t size)
{
    return malloc(size); // no null test, no throw
}
```

- **[new.delete.single]**: a replacement returns non-null or throws. Null is the nothrow_t overload's contract.
- **[expr.new]**: a new-expression tests for null only when the allocation function is declared non-throwing. This one is not.
- So on an exhausted heap, new Foo runs the constructor at address 0 — silently on readable flash, a precise BusFault on the reference U5 (examples/06). Only -fcheck-new stood in the way.

On Zephyr this is a correctness fix, not a discipline one

The default operator new returns null — and the standard says it may not

The Zephyr defaults under CONFIG_CPP=y

Symbol	Default
STD_CPP*	C++11 (C moved to C17; C++ did not)
LIBCPP_IMPLEMENTATION	MINIMAL_LIBCPP — no STL at all
-fno-exceptions -fno-rtti	always, unless a full library is chosen
-fcheck-new	always — the only thing between a null new and a silent write

Monday: REQUIRES_FULL_LIBCPP=y, STD_CPP23=y, picolibc (the only libc the SDK ships), COMPILER_OPT="-D_GLIBCXX_ASSERTIONS". Then the twenty definitions.

std::span — and the thing everyone gets wrong about it

A span is size propagation, not bounds checking. Treat span and hardening as one technique.

```
// before — the size is a convention
void parse(const uint8_t* buf, size_t n);
parse(frame, sizeof(frame) / 2); // compiles. wrong.
// after — the size travels with the data
void parse(std::span<const uint8_t> buf);
parse(frame); // deduced, correct
// the extent can live in the type:
void parse_header(std::span<const uint8_t, 4>);
```

- operator[] has a precondition, not a check ([span.elem]). So do .subspan(), .first(), .last().
- The check is _GLIBCXX_ASSERTIONS: +10 B per site. Unhardened span[i] is the raw pointer's 6 bytes.
- Fixed extent: a constant index past it is -Warray-bounds at -Os. Dynamic extent: only the hardened library sees it.

COMPILER EXPLORER — emit-time

Fixed extent can warn at -Os. The hardened accessor adds the check; per-site figures are CE measurements.

NUCLEO U5A5ZJ-Q — run-time

examples/02_span_harden: plain s[4] → 65535; hardened → abort(); +48 B FLASH in aggregate.

C++17 route: gsl::span — checks operator[] and terminates. C++26: P3471 makes the same checks hardened preconditions.

godbolt.org/z/MraE5rac6

The technique is not the mechanism

Two more that everybody writes as if the first half implied the second

```
std::expected<float, SensorError> read_temp();  
read_temp();           // silent at -Wall -Wextra  
  
[[nodiscard]] std::expected<float, SensorError> read_temp();  
read_temp();           // warning: ignoring return value  
// [expected.syn] carries no attribute; libstdc++ adds none.  
// One annotation per fallible API –  
// make it a clang-tidy rule.
```

- expected needs no exceptions. .value() on an error aborts; use has_value(), operator*, error().
- What expected adds: the value and the error cannot be read through one variable by mistake. The compiler caring is your [[nodiscard]].

godbolt.org/z/GcsM88jdG (nodiscard)

The technique is not the mechanism

Two more that everybody writes as if the first half implied the second

```
enum class Millis : std::uint32_t {};  
enum class Ticks  : std::uint32_t {};  
void set_timeout(Millis);  
set_timeout(ticks);           // does not compile  
set_timeout(Millis{ticks});   // C++17 (P0138)  
set_timeout(static_cast<Millis>(ticks)); // C++11 route  
// no operators: ms + 1 does not compile either
```

- For time, the standard already wrote the strong type: `std::chrono::duration`. Scoped enums for the units it has no type for.
- Zero runtime cost. No sanitizer, no analyzer and no borrow checker finds a unit mix-up — this is where disciplined C++ equals any memory-safe language.

no sanitizer, no analyzer, and no borrow checker finds a unit mix-up

constexpr: fail the build, not the board

and the textbook version does not compile on your build

```
constexpr Baud checked_baud(uint32_t hz, uint32_t pclk) {  
    if (hz == 0 || pclk % hz != 0)  
        throw "unrepresentable baud rate";  
    return Baud{hz};  
}  
constexpr auto UART0 = checked_baud(9600, 48'000'000);  
// error: exception handling disabled, use '-fexceptions'  
// - at the DEFINITION. Every call fails, the good one too.
```

godbolt.org/z/KoM463M9b (throw)

constexpr: fail the build, not the board

and the textbook version does not compile on your build

```
void baud_rate_is_not_representable(); // never defined
constexpr Baud checked_baud(uint32_t hz, uint32_t pclk) {
    if (hz == 0 || pclk % hz != 0)
        baud_rate_is_not_representable();
    return Baud{hz};
}
constexpr auto UART0 = checked_baud( 9600, 48'000'000); // ok
constexpr auto UART1 = checked_baud(115200, 48'000'000);
```

- A non-constexpr function cannot appear in a constant expression: the bad branch stops the build, needs no exceptions, costs nothing in flash, and puts your words in the diagnostic.
- C++26 constexpr exceptions (P3068) make the throw form standard — with -fexceptions. Under -fno-exceptions GCC 16 still rejects it at the definition.
- Often a negative cost: the table computed at build time replaces the runtime computation and the startup code. And every library that grows more constexpr (C++20's algorithms, containers) is more code this reaches.

godbolt.org/z/4h43Ev1xv (fixed)

The arena that quietly falls back to the heap

Fixed-capacity containers: ETL today, std::pmr with one argument you must not forget

```
std::array<std::byte, 1024> arena;  
std::pmr::monotonic_buffer_resource pool{  
    arena.data(), arena.size()};  
// upstream = get_default_resource()  
//           = ::operator new → heap, silently  
std::pmr::monotonic_buffer_resource pool{  
    arena.data(), arena.size(),  
    std::pmr::null_memory_resource()};  
// exhaustion = bad_alloc = abort at a known  
// point, never a hidden heap  
std::pmr::vector<Sample> samples{&pool};
```

Visible in the object file

Two-argument form: U

std::pmr::get_default_resource(). Invisible in testing — the arena is big enough until the day it is not. With the twenty definitions it is a link error, by accident.

- ETL (etl::vector<T,N>, etl::string<N>): capacity fixed at compile time, storage where you put it, still links after the heap is gone. std::inplace_vector is libstdc++ 16 — not in this room.
- <algorithm> is not safe wholesale: stable_sort, inplace_merge and stable_partition allocate; std::sort does not.
- TI's tiarmclang ships no std::pmr at all. Across vendor toolchains, ETL travels better.

godbolt.org/z/EW537zMqb

The hazards the abstract machine does not model

Two groups, and they need opposite advice

Outside the abstract machine — no tool

- DMA and buffer lifetime: a second bus master writes memory the language believes only you can see.
- Cache versus DMA (Cortex-M7): the maintenance ritual that corrupts the neighbour on the same line.
- Interrupts: the memory model has no word for one. Rust's Send/Sync reaches it; C++ has nothing.
- **Defence: design. The type system as a fence around what the language cannot see.**

Inside it, UB — and the silicon answers differently

- Null: a readable vector table on most parts — a BusFault on the reference U5. The part decides.
- Misaligned access: ldr tolerates it, ldrd never does — the width decides, not the -O level.
- Stack depth: no hardware limit on Armv7-M; a 512-byte std::array is a 520-byte frame.
- The C library's own heap: printf("%f") reaches malloc on newlib, not on picolibc.
- **Defence: a tool you already have — UBSan, -Wcast-align=strict, -fstack-usage, --wrap=malloc — and knowing that "works on my board" is the hardware being lenient.**

A null dereference: silent, a bus fault, or a compiler trap

Same Armv8-M, two parts, opposite answers at address 0 — know your target

```
// the bootloader idiom:  
// read the initial SP from the vector table  
std::uint32_t sp() {  
    return *reinterpret_cast<const volatile std::uint32_t*>(0);  
}  
// ARM GCC 14.3.0, Cortex-M33  
// -O1:  movs r3, #0 ; ldr r0, [r3] ; bx lr          works  
// -Os:  movs r3, #0 ; ldr r3, [r3] ; .inst 0xdeff  traps  
//      -fisolate-erroneous-paths-dereference, on at -Os/-O2  
// the emitted load: silent on most parts — BusFaults on the U5
```

godbolt.org/z/v1ePxPec4

A null dereference: silent, a bus fault, or a compiler trap

Same Armv8-M, two parts, opposite answers at address 0 — know your target

```
int f(int* p) {  
    int v = *p;  
    if (p == nullptr) return -1;    // deleted from -O1 up  
    return v;  
}  
// -O1:  ldr r0, [r0] ; bx lr    — no compare at all  
// -fno-delete-null-pointer-checks restores the cbz  
// GCC: "assume ... no code or data element  
//       resides at address zero"
```

godbolt.org/z/v1ePxPec4

A null dereference: silent, a bus fault, or a compiler trap

Same Armv8-M, two parts, opposite answers at address 0 — know your target

COMPILER EXPLORER — emit-time

ARM GCC 14.3.0: -O1 keeps the load; -Os can fold the null read into an undefined-instruction trap.

NUCLEO U5A5ZJ-Q — run-time

examples/06: address 0 is unbacked; a null read or write is a precise BusFault.

- Address 0: readable flash on most Cortex-M, so the bug is silent — but not a law. On the reference STM32U5 it is unbacked: a null read or write is a precise BusFault (measured, examples/06).
- For bugs, make it fault: an MPU region at 0 (Zephyr `NULL_POINTER_EXCEPTION_DETECTION` — default `NONE`), or `-fsanitize=null` in trap mode: a `cbz` before every dereference.
- For a legitimate read of 0: `-fno-delete-null-pointer-checks`, or the alias at `0x08000000`. Prefer `=pattern` to `=zero` init for exactly this reason. References remove the null class at an API boundary — and nothing else; they dangle like pointers.

Design null out — don't check it in: put "absent" in the type (`T&` or `optional`), check before the first dereference, and let an MPU region fault address 0 for everything that slips through — because the compiler deletes the check you add after.

godbolt.org/z/v1ePxPec4

A guard has a width, and a C++ frame steps over it

Stack overflow on a part with no hardware limit — and the frames you did not write

```
// ARM GCC 14.3.0, -Os -Wstack-usage=1 -Wvla
void small()    { std::uint32_t x = 0; use(&x); } // 16 B
void big()      { std::array<std::uint8_t, 512> buf; ... } // 520 B
void aligned()  { alignas(64) std::uint8_t buf[64]; ... } // 128 B
void vla(int n){ std::uint8_t buf[n]; ... } // -Wvla,
// "stack usage might be unbounded"
```

The guard is a region, not a line

Zephyr's MPU stack guard is 64 bytes (128 with the FPU context) — sized for the one thing the hardware pushes past it, the exception frame. A 520-byte local jumps clean over it and lands in the next thread's stack. No fault, no canary. MSPLIM catches the SP move; it does not catch a store below SP.

godbolt.org/z/anYPY1Grb

A guard has a width, and a C++ frame steps over it

Stack overflow on a part with no hardware limit — and the frames you did not write

- In C++ the frame is rarely the size of the code you wrote: a returned aggregate that missed elision, a lambda capturing by value, a chain of inlined calls merged into one frame.
- Static bound in CI: -fstack-usage per function; treat "dynamic" without "bounded" as a failure; -fcallgraph-info gives every virtual and function-pointer call an edge to `__indirect_call` — treat that as unknown. That is the price interfaces and dependency injection pay here, and it is a known price.
- Then the runtime guard (MSPLIM on Armv8-M Mainline, MPU guard elsewhere) and high-water telemetry. All three; none is a substitute.

Measure the frame — the guard is narrower than your C++ says it is.

godbolt.org/z/anYPY1Grb

DMA: It will handle everything for me!

A compiler barrier where a hardware barrier is required

```
buf[0] = 1;    // fill
std::atomic_signal_fence(std::memory_order_release); // ?
DMA_START = 1; // go
// strb r3,[r2] ; str r3,[r2] - identical to NO fence

std::atomic_thread_fence(std::memory_order_release);
// strb r3,[r2] ; dmb ish ; str r3,[r2]
// AN321: "the DMB instruction is required
//         before starting DMA"
```

And the buffer itself

```
void read_sensor() {
    uint8_t buf[64]; // stack
    HAL_UART_Receive_DMA(
        &h, buf, 64);
} // frame pops;
// the engine keeps writing
```

No warning, no tool: the CPU never touches the memory. Who owns that buffer? Something the type system cannot see.

- `std::atomic_signal_fence` is a **compiler-only** barrier — it stops the *compiler* reordering, but emits **no instruction**. It exists to order a thread's accesses against a **signal handler running on the same core**, where no hardware barrier is needed.
- `std::atomic_thread_fence` is a **compiler + hardware** barrier — on Cortex-M it emits `dmb ish`.

godbolt.org/z/6Gj636YjY

DMA: It will handle everything for me!

A compiler barrier where a hardware barrier is required

- The signal fence orders your accesses against a signal handler in the same thread. A DMA engine is on the bus. That is the case the thread fence exists for.
- The C++ shape of a fix: buffers from static storage or a pool; a move-only Transfer that takes the peripheral by value, so a second transfer is a compile error; a destructor that aborts the stream; receive into `std::byte` and make the type the last step.
- Rust solved this with a 'static bound, not the borrow checker. Same conclusion, opposite reasoning.

The fence that emits nothing, and the buffer that outlives its frame — DMA is on the bus, so fence it in hardware and own its buffer by design.

godbolt.org/z/6Gj636YjY

volatile is not synchronization

std::atomic for the interrupt the memory model has no word for

```
volatile std::uint32_t counter;           // does NOT make this safe
void isr() { counter++; }                // load, add, store
void main_loop() { counter++; }           // ISR in between: lost

std::atomic<std::uint32_t> counter;
void isr() { counter.fetch_add(1, std::memory_order_relaxed); }
// M33: ldrex / adds / strex loop - no library, no lock
```

Which fence, for whom

ISR versus main loop:
std::atomic_signal_fence — a compiler barrier, no instruction on Cortex-M, and exactly right.

DMA or a second core:
std::atomic_thread_fence — dmb ish. The previous slide.

What it guarantees: the access *happens* — the compiler won't optimise away, coalesce, or reorder-relative-to-other-volatiles a `volatile` read or write

godbolt.org/z/dlbv6araK (Cortex-M0)

volatile is not synchronization

std::atomic for the interrupt the memory model has no word for

```
static_assert(std::atomic<std::uint32_t>::is_always_lock_free,  
             "not lock-free on this core");  
// Cortex-M0: static assertion failed. Without it:  
//   fetch_add → bl __atomic_fetch_add_4 → undefined reference  
// arm-none-eabi ships no libatomic. Armv6-M has no LDREX/STREX;  
// the M23 does. The boundary is the M0/M0+, not "Baseline".
```

volatile guarantees the access happens. It guarantees nothing about atomicity, ordering against non-volatile accesses, or mutual exclusion — and the standard says even the access is implementation-defined. `REG |= MASK` is a read and a write, which is fatal on a write-1-to-clear register.

godbolt.org/z/dlbv6araK (Cortex-M0)

RAII for hardware — and the standard already wrote the guard

The destructor is the release you skipped on the early-return path

```
SpiLock lock{bus};           // ctor acquires
if (!probe()) return NoDevice; // dtor releases — every path
transfer(...);               // dtor releases at scope end

// the gotcha: a guard's dtor RESTORES, never "enables"
CriticalSection() : key_{irq_lock()} {} // capture
~CriticalSection() { irq_unlock(key_); } // write it back
```

- Zero cost — the destructor is the unlock you would have written, on the early-return path too.

The classic firmware bug: early return that skips the release

RAII gives you the release you can't skip; the standard gives you the guard; you supply only the *restore*.

Layer 2 scorecard

Always the same way: the technique plus its mechanism.

Technique	Code churn	Cost on target	Bug class
Define all 20 operator new/delete	one file	≤ 0 B after --gc-sections	accidental heap use
std::span PLUS _GLIBCXX_ASSERTIONS	new APIs first	+10 B per checked site	out-of-bounds
std::expected PLUS [[nodiscard]] on your function	new APIs first	comparable to an int + out-param	unchecked error paths
std::chrono::duration	new APIs first	0 B	unit-of-measure
constexpr without throw	config code	often negative	bad config found late

Layer 2 scorecard — continued

Four more rows — the same shape: the technique plus its mechanism.

Technique	Code churn	Cost on target	Bug class
<code>std::atomic + the right fence + is_always_lock_free</code>	every shared variable	ldrex/strex; a link error on M0	lost update between ISR and thread
<code>ETL · std::pmr arena PLUS null_memory_resource()</code>	container swap	unmeasured on the board	overflow, hidden heap
<code>DMA: thread fence, static buffers, move-only Transfer</code>	driver API	one dmb ish per start	use-after-scope by a bus master
<code>-fstack-usage bound in CI + a guard + telemetry</code>	none	0 B for the bound	silent overflow into the next thread

"Not 'trust the feature' — verify the mechanism, then make it process."

The good news:

We can do something about this.

VERIFY

read the actual
mechanism

CONSTRAIN

make bad states harder

MEASURE

cost it on target

OWN FAILURE

decide what happens
next

The pattern is not “trust the feature.”

Verify the mechanism — then make it part of the engineering process.

Layer 3

C++26: three new names for checks, three new implementation-defined switches

None of it is "recompile and win". The standard now has a name for each check, and your implementation decides whether it happens — ask the implementation, not the version number.

≈ 7 min

C++17 → C++26: what arrived, and what your toolchain has

Revision · what each standard gives you here · is it on GCC 14.3 (the Zephyr SDK)?

Std	Arrival	On GCC 14.3?
C++17	over-aligned new (the 20th allocation function) · std::pmr arenas · [[nodiscard]] · if constexpr · enum from underlying type	yes — the floor most vendor compilers stop at
C++20	std::span · std::bit_cast (the punning idiom without memcpy) · constinit (no static-init brick) · consteval · concepts · std::atomic_ref	yes, complete for everything
C++23	std::expected · std::start_lifetime_as (DMA buffers without a copy) · compound assignment on volatile un-deprecated · a larger freestanding subset (P1642)	expected yes · start_lifetime_as no (GCC 16)
C++26	hardened preconditions (P3471) · erroneous behaviour (P2795) · contracts (P2900) · std::inplace_vector (P0843) · constexpr exceptions (P3068) · freestanding span/expected	no · no · no · no · no · yes (libstdc++ 14)

The feature-test macro, not `__cplusplus`, is what tells you which of these you actually have.

What shipped in March 2026 — and what you can actually build

The version gap is a distribution gap: the checks are standard, the compilers are one generation away

C++26 feature	The switch the standard leaves to you	GCC / libstdc++	On GCC 14.3 (Zephyr SDK)?
Contracts — pre, post, contract_assert (P2900R14)	which of ignore / observe / enforce / quick-enforce an assertion gets: implementation-defined	GCC 16, only with -fcontracts	NO
Hardened preconditions (P3471R4)	"implementation-defined whether the implementation is a hardened implementation"; a -D macro "would also be conforming"	libstdc++ 15, partial	NO as a standard feature — YES as the macro, today
Erroneous behaviour (P2795R5)	"recommended to diagnose" — recommended, not required; zero by default on GCC	GCC 16	NO — the flag, YES: -ftrivial-auto-var-init=pattern
std::inplace_vector (P0843R14)	under -fno-exceptions: try_push_back, the freestanding API	libstdc++ 16	NO — ETL today

Profiles are in C++26? They are not — rerouted to C++29 (P3589, P3984)

The pre-standard form of each check ships this week

Plan for C++26. Do not plan on it.

Check	GCC / libstdc++ today	Clang / libc++ today
Library bounds checks	<code>-D_GLIBCXX_ASSERTIONS</code>	<code>-D_LIBCPP_HARDENING_MODE=..._FAST</code>
Initialized automatics	<code>-ftrivial-auto-var-init=pattern</code>	same flag
std::expected	<code>libstdc++ 12</code>	<code>libc++ 16</code>
Fixed-capacity vector	ETL	ETL
Preconditions as code	<code>[[nodiscard]]</code> on your functions, <code>constexpr</code> where the value is static	same

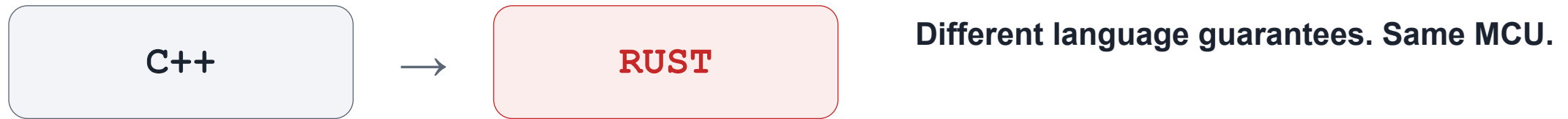
- `-ffreestanding` on your C++ TUs (GCC 13+) bans `<vector>`, `<string>`, `<map>`, `<memory_resource>` at compile time — while `<span>`, `<expected>`, `<optional>`, `<algorithm>` keep compiling. It implies `-fno-builtin` (spell the punning idiom `std::bit_cast`) and takes `<chrono>` with it. Zephyr passes it to C only.

PACBTI, the one new hardware mitigation: control-flow integrity, not memory safety; optional on M52/M85; on the M33 in this room `-mbranch-protection` assembles to NOPs — flash spent, nothing protected.

godbolt.org/z/sd5Mjq6T8 (`-ffreestanding` vs the `memcpy` fold)

What if we change the language?

Rust changes the language-level guarantees. The hardware seam is still ours to own.



Rust

An honest embedded perspective — five minutes, then we move on

Concede the wins in full, name the costs on this board, and say what the seam is made of.

≈ 5 min

Where it wins — conceded in full

Say the mechanism, because the room will ask

- Temporal safety is a property of the type system: use-after-free, use-after-scope, double free rejected at compile time, on every path, nothing in the image. GCC's `-Wuse-after-free` and Clang 23's `-Wlifetime-safety` are static too — per-TU heuristics that warn. Property versus best effort. That is the real advantage.
- Send/Sync turns ISR-versus-main-loop sharing into a type error. C++ has no equivalent.
- Spatial safety is a run-time bounds check that panics: the same `cmp-branch-trap` shape as a hardened `span::operator[]`. What Rust adds is that it is on by default, not that it is free. Their check is the one you can turn on this week.
- Android: memory-safety bugs from 76 % (2019) to below 20 % (2025), by writing new code in a memory-safe language.

Where it stands on this board — September 2026

Supported and qualified are different words

- Every Cortex-M target is Tier 2: "guaranteed to build", tests "not always run". No bare-metal target is Tier 1.
- Ferrocene 26.05 is qualified ASIL D / SIL 3 — for the Cortex-M4/M7 targets. thumbv8m.main (this board) is "supported": "suitable for use outside of safety-critical contexts". The certified core subset is ASIL B / SIL 2.
- Zephyr's Rust module: "rather minimalistic", 8 platforms, 4 of them QEMU, not this board.
- Footprint cuts both ways. On an STM32U585 — this series — Rust was +7.4 KB ROM and -20 KB RAM: the C build carried a 25.6 KB heap, the Rust build none.
- Still yours in either language: DMA lifetime, the cache, registers, stack depth. Integer overflow checks are off in a Rust release build by default.

The pragmatic middle ground — and what the seam is made of

New isolated module in Rust, existing C++ stays. The seam is C.

- What crosses extern "C": `#[repr(C)]` structs, raw pointers, integers. Not `Vec` or slices — and not `std::span`, `std::expected` or your strong types. Every Layer 2 technique stops at the seam and is rebuilt on the far side; the seam is the interface Layer 2 spent nineteen minutes replacing.
- `cxx` carries richer types across, but needs Rust's `alloc`: "`no_alloc` is incomplete and `semver` exempt". With the twenty definitions in place it is off the table.
- **Three decisions to make on purpose:**
- Unwinding: `thumbv*-none-eabi` is already `panic=abort`; C++ exceptions into Rust are UB; `-fno-exceptions` makes the triple consistent.
- Allocation: Rust's `alloc` goes to `#[global_allocator]`, never operator `new` — the twenty definitions do not see it.
- Who links last: corrosion; one linker script, and it is yours.

The cautionary tale — about the seam, not the language

curl removed its Rust hyper backend in December 2024 at "perhaps 95 % of the way", tests passing. Stenberg's reasons: "there simply were no users asking for it" and "almost no developers interested or knowledgeable enough to work on it" — the glue needed people fluent in both languages. curl kept `rustls` and `quiche`, the Rust it could staff.

For a new isolated module on a supported target, Rust is defensible today. For 200k certified lines on a vendor toolchain, it is not. Both true at once; the seam between them is C.

Checklist

Putting it together: the order of work, and the entries that lie to you

Ordered by effort-to-impact. Nothing before Year 1 needs a rewrite, a new language or a budget line.

≈ 6 min

Week 0 — decide what a trap does on your device

Every Week 1 item converts silent corruption into a termination. On Zephyr they all end in the same weak C function.

Mechanism	Where it ends
hardened library violation	bl abort → printk("abort()") → k_panic()
UBSan trap	udf → UsageFault (Mainline) / HardFault (Baseline)
stack canary	__stack_chk_fail → z_except_reason(K_ERR_STACK_CHK_FAIL)
stack limit / MPU guard	a fault
→ all four	k_sys_fatal_error_handler() — default: lock interrupts and spin

Own that handler before the first trap ships. Safe state, then reset, log or halt — in the safety case, not the compiler flags. If certified: MISRA C++:2023 Rule 18.5.2 ("program-terminating functions should not be used", Advisory) is the re-categorisation plan that covers every trap at once — and its rationale is the one place in MISRA that agrees with slide 6.

"does this make the device crash more?" Yes. On purpose.

Week 1 — build configuration, plus one new file. No changes to existing source.

- Fix the C++ defaults: `REQUIRES_FULL_LIBCPP`, `STD_CPP23`, `picolibc` — the macro has no header to read otherwise
- `ninja -t` commands: read what you already pass
- The twenty `operator new/delete` definitions — first; the link errors are your allocation inventory
- `-Wextra`, `-Wsign-conversion` by name, `-Werror`; baseline; zero new
- `-D_GLIBCXX_ASSERTIONS` (or the `libc++` macro) — globally
- `HW_STACK_PROTECTION` — on this board the `defconfig` already did it; check yours
- `STACK_CANARIES_STRONG`, not `STACK_CANARIES`
- UBSan as two builds, plus the CXX line
- `-ftrivial-auto-var-init=pattern`, `-Wtrivial-auto-var-init`
- Null-pointer detection: the default is `NONE`
- west build `-t hardenconfig` — read it as a prompt, not a score

The checklist that lies to you

Entries that read as a mitigation and are not — no error, no warning, no line in the build log

The entry	What it reads as	What it does
<code>CONFIG_UBSAN=y</code>	"Build with UBSan"	C only. Runtime linked, image bigger, your C++ unchecked
<code>-fsanitize=undefined</code> with <code>-fno-rtti</code>	the whole sanitizer	vp _{tr} — the one C++ object-identity check — dropped silently
<code>CONFIG_STACK_CANARIES=y</code>	"buffers larger than 8 bytes"	character arrays only: <code>std::byte[64]</code> , <code>uint32_t[16]</code> unguarded
<code>-fhardened</code>	GCC's hardening umbrella	on arm-none-eabi: a warning, and the entire umbrella off
<code>CONFIG_FORTIFY_SOURCE_COMPILE_TIME=y</code>	"Detect buffer overflows in libc calls"	the GCC property is empty; only <code>_RUN_TIME</code> defines the macro
<code>-D_GLIBCXX_ASSERTIONS</code> on a libc++ build	the hardened library	a macro nobody reads

The checklist that lies to you

Entries that read as a mitigation and are not — no error, no warning, no line in the build log

The entry	What it reads as	What it does
<code>std::span<T></code>	a bounds-checked view	a precondition, not a check
<code>std::expected<T,E> f();</code>	an error you cannot ignore	no <code>[[nodiscard]]</code> on the type
<code>monotonic_buffer_resource{buf, n}</code>	a heap-free arena	falls back to <code>::operator new</code> when full
<code>void* operator new(size_t) = delete;</code>	a heap ban	ill-formed; links through on GCC
<code>std::atomic_signal_fence</code> before DMA	a memory barrier	no instruction at all

Three probes

```
ninja -t commands | grep fsanitize
nm -C zephyr.elf | grep -E '_Znwj|malloc'
```

one span accessor in `-S`, with and without the macro

Presenting it upstairs

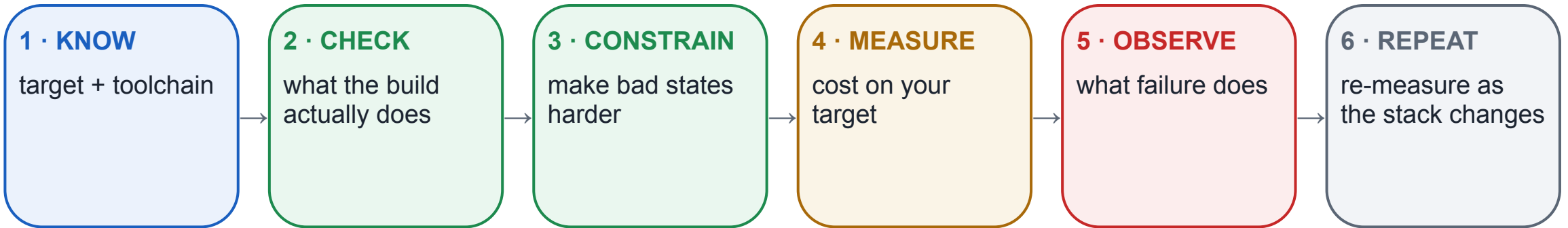
The only dated ask was a published roadmap. This is one — if you answer the awkward row.

CISA's six roadmap elements	What you now have	What you still write
Defined phases with dates and outcomes	Week 0 → Year 1	the dates, and the measured numbers
Date for memory-safe languages in new systems	Rust: where it wins, what it costs on this part	an explicit answer: "not yet, reviewed annually, here is why"
Developer training and integration plan	Layers 1–3	who reads it, when
External dependency plan (C and C++ libraries)	the twenty definitions, --wrap=malloc	the list they produce
Transparency plan	—	a cadence: re-measure quarterly
CVE support program plan	5.5 % of Project Memoria vendors ever patched	the commitment — CRA Art. 14 makes it a legal one

Say it in the three-axis form: "zero code change, ten bytes per checked access, prevents out-of-bounds writes". Use Android's curve for prioritisation — new code first. Let the measurements carry the cost.

The embedded memory-safety loop

A repeatable engineering habit, not a one-time checklist.



Know → Check → Constrain → Measure → Observe → Repeat

That is the roadmap. The checklist is only the current snapshot of it.

Find me and take it all with you



Darijo Topić

*Embedded Software Engineer
Mettler Toledo
Zürich, Switzerland*

topic.darijo@gmail.com

<https://www.linkedin.com/in/darijo-topić-8bb3484b/>

Everything is in the repo

github.com/logistm/CppCon26MemorySafetyEmbeddedMicrocontroller

Everything in this talk is reproducible from your seat
Reach out
I answer embedded C++ questions.

Sources: github.com/logistm/CppCon26MemorySafetyEmbeddedMicrocontroller

Q&A

